AuctionHub: Software Test Documentation

Test strategy, coverage matrix and curated test case catalogue

Project	AuctionHub Online Auction Platform
Team code	FYP-26-S2-24
Document	Software Test Documentation for the Final Technical Document (FTD)
Date	6 August 2026
Automated tests	1,738 total: 1,548 backend and 190 frontend, 0 failing
Cases documented	124, spanning all 16 functional areas

Contents

Team FYP-26-S2-24, AuctionHub Online Auction Platform. This document sets out the test strategy, the section by section coverage matrix, and the curated test case catalogue for the Final Technical Document (FTD). It was generated from the live repository, and every case listed below maps to a real test method that executes.

1. Summary of Automated Testing

1.1 Headline numbers (safe to quote verbatim in slides)

Metric	Value
Total automated tests	**1,738**
Backend (Java) tests executed	**1,548**
Frontend (React) tests executed	**190**
Backend test classes	122
Backend test source files	123 (122 test classes plus 1 shared test helper)
Frontend test files	17
Declared backend test methods	1,443, which expands to 1,548 executions via parameterised tests
Tests failing	**0**
Tests skipped	6 (environment-gated live-database integration tests)
Backend suite runtime	about 7 s
Frontend suite runtime	about 2.3 s
Functional sections covered	16 of 16
Test cases documented in this report	**124**

Slide-ready one-liner: *"AuctionHub is covered by 1,738 automated tests: 1,548 backend (JUnit 5 with Mockito) and 190 frontend (Vitest with React Testing Library). All pass, and the two suites together run in under 10 seconds. This document presents 124 representative cases spanning all 16 functional areas of the system."*

1.2 Frameworks and tooling

Layer	Framework	Version	Notes
Backend unit/integration	JUnit 5 (Jupiter)	5.10.2	`junit-jupiter-api`, `-engine`, `-params`
Backend mocking	Mockito	5.23.0	Includes `mockStatic` for static utility isolation
Backend build/runner	Apache Maven with Surefire		`mvn test`
Backend runtime under test	Jakarta Servlet API	6.0.0	Servlets tested with mocked request/response
Database (mocked)	PostgreSQL JDBC	42.7.10	JDBC layer mocked, so no live DB is required
Frontend test runner	Vitest	4.1.10	`vitest run`
Frontend component testing	React Testing Library	16.3.2	with `@testing-library/jest-dom`, `user-event`
Frontend DOM environment	jsdom	29.1.1	

1.3 Commands to reproduce

```
# Backend, 1,548 tests
cd FYP
mvn -B test

# Frontend, 190 tests
cd FYP/Frontend
npm install # first run only
npm test # alias for: vitest run
```

To enable the 6 environment-gated live-database integration tests:

```
cd FYP
AUCTION_DB_IT=true mvn -B test # requires a reachable PostgreSQL instance
```

1.4 Testing approach

The suite is written at the unit and service-integration level rather than as end-to-end browser automation. Servlets are exercised through mocked `HttpServletRequest`, `HttpServletResponse` and `HttpSession` objects. DAOs are exercised through mocked `Connection`, `PreparedStatement` and `ResultSet` objects. This has three consequences worth stating:

- The suite runs deterministically in CI with no database, no Tomcat and no network dependency.
- Assertions can check the response and also the exact SQL parameters that were bound, which makes ownership scoping and injection-safety provable rather than assumed.
- Two integration test classes, `AdminManagementDAOIntegrationTest` and `DatabaseBackupRestoreIntegrationTest`, run against a real PostgreSQL instance. They are gated behind the `AUCTION_DB_IT` environment variable so the default build stays hermetic. These are the 6 tests reported as skipped.

End-to-end user journeys, visual layout, email delivery over SMTP, and Telegram message rendering in the real client were verified by manual testing. See Section 5.

2. Section-by-Section Coverage Matrix

Every backend test class is assigned to exactly one section, so the counts below sum to the 1,443 declared backend test methods.

#	Website section / module	Test classes	Declared tests	In this report	Coverage assessment
1	Guest landing page and dynamic content	5	27	4	**Strong.** Content is DB-driven and admin-editable. Caching and XSS rejection are covered
2	Search, filters, sorting and browse	5	80	5	**Strong.** Heavy negative and injection coverage on every filter and sort key
3	Auction detail view and public seller profile	4	51	4	**Adequate.** Masking, pagination and state logic covered. See gap G2
4	Registration and login	5	58	6	**Strong.** Includes brute-force lockout and role-escalation rejection
5	Password reset, OTP and two-factor authentication	6	57	6	**Strong.** Expiry, attempt limits and enumeration resistance all covered
6	Session, RBAC, filters and platform security	8	43	7	**Strong.** RBAC matrix, CSP headers, PDPA encryption and masking
7	Bidding: ascending, Dutch, Buy It Now, blind	10	130	33	**Strong.** All three auction types covered end to end, with a regression pinned on every read path that can reach a sealed bid
8	Auto-bid and proxy bidding	5	45	4	**Strong.** Competition, tie-break and ceiling logic covered
9	Buyer engagement: watchlist, Q&A, ratings, reports	11	139	7	**Strong.** Largest single block of tests
10	Orders, payment, shipping and refunds	7	57	6	**Strong.** Includes unpaid-order auto-cancellation
11	Seller: listings, maintenance and analytics	9	215	7	**Strong.** The most heavily tested section
12	Admin console and database backup/restore	11	133	7	**Adequate.** Legacy servlets fully covered. See gap G3
13	Recommendation engine	4	104	6	**Strong.** Pipeline, CF maths, diversity cap, explainability
14	Notifications (in-app) and preferences	9	93	5	**Strong.** Dedupe, opt-out gating and PII-leak checks
15	Telegram notification integration	10	121	5	**Strong.** Webhook auth, brute force, retry ladder, outbox worker
16	Account management, PDPA and account closure	13	90	4	**Strong.** Encryption round-trip, anonymisation, rollback
	Frontend (React SPA)	17 files	190	8	**Partial.** Utilities, hooks and API layer strong. Page components thin, see gap G7
	TOTAL	**122 + 17**	**1,443 + 190**	**124**	

2.1 Coverage by test type (curated set)

Type	Count	Share
Security / access control	52	42%
Happy path / functional	36	29%
Negative / validation	21	17%
Boundary / precision	10	8%
Integration / lifecycle	5	4%
Total	**124**	100%

71% of the documented cases are security, negative, boundary or lifecycle cases, and only 29% are happy path. This split is deliberate. A catalogue that is only happy path does not evidence engineering judgement, and Session 2 assessment weights security heavily.

3. Curated Test Case Catalogue

3.1 How to read these tables

- **ID** is a stable reference for the FTD and for traceability back to requirements.
- **Test class and method** is the exact, real, executing code. An assessor can open the file and run it.
- **Type** is one of `Functional` (happy path), `Negative`, `Security`, `Boundary`, `Integration`.

All backend classes live under `FYP/src/test/java/`. All frontend tests live under `FYP/Frontend/src/`.

3.2 Section 1: Guest Landing Page and Dynamic Content

Addresses assessor feedback that landing content must be dynamic, not hardcoded.

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-GST-001	`TestLandingContentApiServlet.returnsContentMap`	Landing page copy is served from the database, not hardcoded in the JSX	Landing content rows exist in `landing_content`	`GET /api/landing-content`	200 with a key/value map of every landing field sourced from the DB	Functional
TC-GST-002	`TestLandingContentApiServlet.failsSoftOnDatabaseError`	The landing page still renders if the content query fails	Database throws on read	`GET /api/landing-content`	Empty content map returned and no 500. The page degrades to defaults instead of breaking for guests	Negative
TC-GST-003	`TestAdminLandingContentApiServlet.savesSubmittedFields`	An admin can edit landing content without a code change or redeploy	Session holds an ADMIN user	`POST` with valid content keys and values	Values persisted and the public content cache invalidated	Functional
TC-GST-004	`TestAdminLandingContentApiServlet.rejectsMarkup`	Admin-authored landing copy cannot inject HTML or script into the public page	Session holds an ADMIN user	Content value containing HTML markup	400 rejection, and the value is not persisted	Security

3.3 Section 2: Search, Filters, Sorting and Browse

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-SRC-001	`TestSearchServlet.testValidKeywordReturnsResults`	A guest can search listings by keyword	Active listings exist, no session required	`?q=laptop`	Matching listings forwarded to the results view	Functional
TC-SRC-002	`TestSearchServlet.testSqlInjectionAttemptIsHandledSafely`	Search input cannot be used for SQL injection	None	`q=' OR '1'='1`	Treated as a literal keyword via a bound parameter. No rows leaked and no error	Security
TC-SRC-003	`TestSearchServlet.testQueryAtMaxLength`	The keyword length limit accepts exactly the boundary value	None	Keyword of exactly the maximum permitted length	Accepted and searched normally	Boundary
TC-SRC-004	`TestSearchServletSort.testSqlInjectionReturnsDefault`	The sort key cannot be injected into the `ORDER BY` clause	None	`sort=; DROP TABLE auction`	Unrecognised sort silently falls back to the default, and the generated `ORDER BY` contains no user input	Security
TC-SRC-005	`TestSearchServletFilters.testAllFiltersCombined`	Price, condition, location and ending-within filters compose correctly	Listings spanning several prices and conditions	All four filter parameters supplied together	A single `SearchFilter` carrying all four constraints is passed to the DAO	Functional

3.4 Section 3: Auction Detail View and Public Seller Profile

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-DET-001	`TestAuctionBidHistory.testPublicAccessNoSession`	An unregistered visitor can view an auction's bid history	No session	`GET /auction-bid-history?auctionId=1`	200 with bid history, and no redirect to login	Functional
TC-DET-002	`TestAuctionBidHistory.testSecurityUtilMaskingRules`	Bidder identities are masked in the public bid history (PDPA)	Auction has bids from several users	Public bid-history request	Bidder names and emails returned in masked form only	Security
TC-DET-003	`TestAuctionBidHistory.testUnknownAuction404`	A request for a non-existent auction is handled cleanly	None	`auctionId` that does not exist	404 with no stack trace exposed	Negative

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-DET-004	`TestSellerProfileServlet.testMaskedEmailInProfile`	A public seller profile never exposes the seller's real email	Seller with completed sales exists	`GET` public seller profile	Email rendered masked, and the raw address never appears in the response	Security

3.5 Section 4: Registration and Login

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-REG-001	`TestRegisterServlet.testInsert`	A visitor can create an account	Username and email unused	Valid registration form	User row inserted, then redirect to login	Functional
TC-REG-002	`TestRegisterServlet.testSuppliedRoleIsIgnored`	A registrant cannot escalate themselves to ADMIN via a crafted form field	None	Registration form with `role=ADMIN` injected	Supplied role discarded, and the account is created as an ordinary member	Security
TC-REG-003	`TestRegisterServlet.testPasswordValidation`	Weak passwords are refused at registration	None	Password failing the complexity policy	Rejected with a validation error, and no user is created	Negative
TC-REG-004	`TestLoginServlet.testSuccessfulLoginSetsSession`	A registered member can sign in and receive a session	Active account with known password	Correct email and password	Session populated with user id and role, then redirect to the member area	Functional
TC-REG-005	`TestLoginServlet.testSuspendedUserCannotLogin`	A suspended account cannot sign in even with correct credentials	Account status is SUSPENDED	Correct email and password	Login refused with a suspension message, and no session is created	Security
TC-REG-006	`TestAuthApiServlet.loginLocksOutAfterThreshold`	Repeated failed logins trigger a brute-force lockout	Account exists	N consecutive wrong passwords, where N is the configured threshold	Lockout response with remaining cooldown. Further attempts are refused even with the correct password	Security

3.6 Section 5: Password Reset, OTP and Two-Factor Authentication

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-OTP-001	`TestPasswordResetServlet.testSuccessfulResetHashesWithSecurityUtil`	A member can reset a forgotten password via OTP	Valid unexpired OTP issued for the account	Correct OTP and a compliant new password	Password stored as a BCrypt hash via `SecurityUtil`, and the OTP is invalidated	Functional
TC-OTP-002	`TestPasswordResetServlet.testExpiredOtpRejectsReset`	An expired OTP cannot be used	OTP issued and expired	Expired OTP and a new password	Reset refused, and the password is unchanged	Negative
TC-OTP-003	`TestAuthApiServlet.forgotPasswordDoesNotEnumerate`	The reset flow does not reveal whether an email is registered	None	Reset request for an unregistered email	Identical generic response to the registered case, and no OTP is issued	Security
TC-OTP-004	`OtpStoreTest.attemptLimitInvalidatesOtp`	An OTP cannot be brute-forced	Valid OTP issued	Repeated wrong OTP guesses up to the attempt limit	OTP invalidated after the limit, so even the correct code then fails	Security
TC-OTP-005	`TestTwoFactorServlet.testConfirmValidOtpEnablesTwoFactor`	A member can enable TOTP-based 2FA	Authenticated session with a pending 2FA secret	Correct 6-digit TOTP code	2FA enabled and the secret persisted encrypted	Functional
TC-OTP-006	`TestTwoFactorApiServlet.verifyLoginWrongOtp`	A wrong 2FA code at login does not grant access	Password stage passed, 2FA pending	Incorrect TOTP code	401, and the session remains unauthenticated	Security

3.7 Section 6: Session, RBAC, Filters and Platform Security

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-SEC-001	`TestTwoFactorServlet.testNullSessionDeniesAllAccess`	`RbacUtil` denies every role check when there is no session	No session	`hasRole`, `isAdmin`, `isBuyer`, `isSeller` with `null`	All return false. The check fails closed, never open	Security
TC-SEC-002	`TestAdminFilter.TestBuyer`	A non-admin cannot reach any `/admin/*` URL	Session holds a BUYER	`GET /admin/dashboard`	Blocked by `AdminFilter` before the servlet runs	Security
TC-SEC-003	`TestAdminFilter.TestAdmin`	An admin passes the admin filter	Session holds an ADMIN	`GET /admin/dashboard`	Request forwarded down the filter chain	Functional
TC-SEC-004	`TestLogoutServlet.testProtectedPageBlocksRequestAfterSessionInvalidation`	A session cannot be reused after logout	Member logged in, then logs out	Replay of a protected request with the stale session	Access denied, and the user is redirected to login	Security
TC-SEC-005	`TestSecurityFilter.securityHeaders`	Every response carries the hardening headers	None	Any request	`Content-Security-Policy`, `X-Content-Type-Options`, `X-Frame-Options` and related headers present	Security
TC-SEC-006	`SecurityUtilTest.encryptUsesRandomIv`	PDPA-protected fields are encrypted with a fresh IV each time	None	Encrypt the same plaintext twice	Two different ciphertexts, both decrypting to the original. No deterministic-encryption leak	Security
TC-SEC-007	`SecurityUtilTest.sanitizeXss`	User-supplied text is neutralised before storage or echo	None	`<script>alert(1)</script>`	Markup escaped, and no executable script survives	Security

3.8 Section 7: Bidding (Ascending, Dutch, Buy It Now, Blind)

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BID-001	`TestPlaceBidServlet.successfulBid`	A buyer can place a valid bid on an ascending (`PRICE_UP`) auction	Auction live, and the bid exceeds the current price	Valid `auctionId` and amount	Bid recorded, current price updated, success message returned	Functional
TC-BID-002	`TestPlaceBidServlet.selfBidRejected`	A seller cannot bid on their own listing	Session user owns the auction	Valid bid amount	Bid refused with an explanatory message	Security
TC-BID-003	`TestPlaceBidServlet.equalBidRejected`	A bid equal to the current price is not accepted	Current price is X	Bid of exactly X	Refused, because the price must strictly increase	Boundary
TC-BID-004	`TestPlaceBidServlet.negativeBidAmount`	A negative bid is rejected before reaching the database	Auction live	Amount of `-50`	Validation error, and no DAO call is made	Negative
TC-BID-005	`TestPlaceBidServlet.closedAuction`	Bidding on an ended auction is refused	Auction end time has passed	Valid amount	Refused with a message saying the auction is closed	Negative

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BID-006	`DutchClockTest.listedPriceFollowsTheClock`	A Dutch auction's displayed price falls over time according to the clock	Dutch auction with start price, floor price, start and end times	Query price at start, midpoint and end	Price equals the start price at t0 and the linear midpoint halfway through, and never drops below the floor	Functional
TC-BID-007	`TestBidApiServlet.bidTooLow`	The SPA bidding API rejects an under-minimum bid	Authenticated buyer, live auction	Amount below the required increment	400 with the minimum acceptable bid returned	Negative
TC-BID-008	`BidDAORateLimitTest.secondBidInsideWindowRejected`	Bid spamming is rate limited per user per auction	User placed a bid moments ago	Second bid inside the cooldown window	Rejected, and the DAO query is scoped to that user and auction only	Security

Blind (sealed-bid) auctions

The defining property of a blind auction is that no bidder can see what anyone else has bid while it is open. That property is not one check in one place. The standing bid is derivable from the detail payload, the bid-history endpoint, the live SSE snapshot, and every listing projection that computes a price from `MAX(bid_amount)`. TC-BLD-001 to TC-BLD-008 pin the guard on each of those read paths, and the rest cover the write side and the close.

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BLD-001	`TestBlindAuction.openBlindHidesTheStandingBidFromRivalBidders`	A rival bidder cannot see the leading sealed bid while the auction is live	Live blind auction with 3 sealed bids, leading bid $250	`GET /api/auction/{id}` as a signed-in buyer	`currentBid` is `null` and `sealed` is `true`. Only the bid **count** is returned	Security
TC-BLD-002	`TestBlindAuction.openBlindHidesTheStandingBidFromAnonymousVisitors`	An unregistered visitor cannot see the leading sealed bid either	Same auction, no session	`GET /api/auction/{id}` with no auth token	`currentBid` is `null`, and `isOwner` is `false`	Security
TC-BLD-003	`TestBlindAuction.theSellerSeesTheStandingBidOnTheirOwnListing`	The seller does see the standing bid, because "Declare Winner Early" sells at that price	Live blind auction owned by the viewer	`GET /api/auction/{id}` as the owner	`currentBid` is $250, `sealed` is `false`, `isOwner` is `true`	Functional
TC-BLD-004	`TestBlindAuction.closedBlindRevealsTheWinningBid`	The winning amount is revealed to everyone once the auction closes	Blind auction past its end time	`GET /api/auction/{id}` as a buyer	`currentBid` is $250, and `sealed` is `false`	Functional
TC-BLD-005	`TestBlindAuction.aBidderSeesTheirOwnBidButNotTheOthers`	A bidder can see their own sealed bid and nobody else's	Viewer has submitted a $120 sealed bid, and the leading bid is $250	`GET /api/auction/{id}` as that bidder	`mySealedBidAmount` is $120 while `currentBid` stays `null`	Security
TC-BLD-006	`TestBlindAuction.openBlindBidHistoryReturnsNoRows`	The bid-history endpoint returns no rows while the auction is sealed	Live blind auction with 3 bids	`GET /api/auction/{id}/bids`	Empty `bids` array, `sealed: true`, total of 3, and the history DAO is never called	Security
TC-BLD-007	`AuctionEventPublisherBlindTest.openBlindSnapshotCarriesNoAmount`	The live SSE price broadcast carries no amount for an open blind auction	Live blind auction, leading bid $250	Build the broadcast snapshot	`currentBid` is `null`, and `numBids` is still published	Security
TC-BLD-008	`TestBlindAuction.searchResolvesABlindListingToItsStartingPrice`, with `thePriceFilterCannotProbeTheSealedBid`, `trendingResolvesABlindListingToItsStartingPrice` and `featuredResolvesABlindListingToItsStartingPrice`	Every listing projection resolves a blind row to its entry price, so no card, strip or price filter can be used to probe the sealed bid	Search, price-filtered search, trending and featured queries	Execute each query and capture the SQL	Every projected price column carries the guard that resolves `auction_type = 3` to `starting_price`	Security

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BLD-009	`TestBlindAuction.aSealedBidIsAcknowledgedWithoutRevealingAnything`	A buyer can submit a sealed bid and is told only that it was received	Authenticated buyer, live blind auction	`POST /api/bid` with $250	200, and the confirmation names the mechanism without echoing any price back	Functional
TC-BLD-010	`TestBlindAuction.aSecondSealedBidFromTheSameBuyerIsRefused`	One sealed bid per buyer, so a bidder cannot revise upward after the fact	Buyer already has a sealed bid on this auction	`POST /api/bid` with a higher amount	400 "You have already submitted a sealed bid for this auction."	Negative
TC-BLD-011	`TestBlindAuction.aBidAtExactlyTheEntryPriceIsAccepted` with `.aBidOneCentUnderTheEntryPriceIsRefused`	The entry price is an inclusive floor, to the cent	Blind auction with a $100 starting price	Bids of $100.00 and $99.99	$100.00 accepted. $99.99 refused as too low, and nothing is committed	Boundary
TC-BLD-012	`TestBlindAuction.buyItNowDoesNotApplyToABlindAuction` with `.dutchAcceptanceDoesNotApplyToABlindAuction`	Mechanics that belong to other auction types are refused on a blind listing	Live blind auction	Buy It Now purchase, then Dutch clock acceptance	Both refused with `WRONG_AUCTION_TYPE`	Negative
TC-BLD-013	`TestBlindAuction.aSealedBidNotifiesNoOutbidBidder`	No ascending-style outbid or new-bid notification is sent, since that alone would leak the standing bid	Live blind auction with existing sealed bids	A new sealed bid succeeds	Neither `notifyOutbid` nor `notifySellerNewBid` is called	Security
TC-BLD-014	`TestBlindAuction.theHighestSealedBidWins`, `.theWinnersOrderIsRaisedForTheSealedBid`, `.theWinnerAndEveryLoserAreTold`	At close the highest sealed bid wins, the order is raised for exactly that amount, and the result is announced	Expired blind auction, top sealed bid $250 by bidder 7	Finalise the auction	Winner is bidder 7, `winning_bid` is $250.00, one order is raised for $250.00, the winner is notified WON and every other bidder is notified LOST	Integration

Two supporting behaviours are worth naming under questioning. Losing sealed bids are never promoted onto the auction record (`losingSealedBidsAreNeverWritten`), and the winner query ranks by `bid_amount DESC, bid_time ASC`, so an exact tie is settled in favour of whoever bid first (`tiesAreBrokenByWhoBidFirst`).

Blind auctions: confidentiality regressions

The cases above pin the guard where it was already correct. Writing them prompted a full sweep of every server path that projects a price or a bid amount, which found **five** paths that could reach a live blind auction's leading bid. Those are documented in Section 5.1. All five are fixed, and the cases below exist to stop them reopening. Each one was run against the pre-fix code and failed. They are the strongest material in this document, because each is tied to a defect that actually existed rather than to an assumption about the code.

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BLD-015	`TestBlindAuction.watchlistDoesNotLeakTheSealedBid`	Watchlisting a live blind auction does not reveal its leading bid, which fixes D1	Buyer with a live blind auction on their watchlist	`WatchlistDAO.listByUser`, capturing the SQL	Every price computed from `MAX(b.bid_amount)` carries the `auction_type = 3` guard	Security
TC-BLD-016	`TestBlindAuction.sellerProfileDoesNotLeakTheSealedBid`	A public seller profile does not reveal it to a visitor with no session, which fixes D2	Seller with a live blind listing	`SellerProfileDAO.getActiveListings`, capturing the SQL	The same guard is present on the projected `current_price`	Security
TC-BLD-017	`TestBlindAuction.bidHistoryHidesLiveSealedBidsForEveryCaller`	The bid-history DAO returns nothing for a live blind auction whichever servlet asks, which fixes D3	Live blind auction with sealed bids	Both `BidDAO.getBidHistory` overloads. The legacy JSP servlets use the three-argument one	Both queries exclude rows belonging to a live blind auction	Security

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BLD-018	`TestBlindAuction.ascendingBidOnABlindAuctionIsRefused`	An ascending bid on a sealed auction is refused outright, so the rejection cannot be used as a price oracle. This fixes D4	Live blind auction, leading sealed bid unknown to the caller	`BidDAO.placeBid` with $250, which is the path the legacy `/protected/bid` servlet takes	`WRONG_AUCTION_TYPE`, the transaction is rolled back and never committed, and no row is inserted into `bids`	Security
TC-BLD-019	`TestBlindAuction.thePriceFilteredCountCannotProbeTheSealedBid`	The search result **count** cannot be used to binary-search the sealed bid, which fixes D5	Search with both a minimum and a maximum price	`SearchDAO.count` with a price filter	The count's inner query uses the same sealed-safe price column as the result page	Security
TC-BLD-020	`TestBlindAuction.aConcludedBlindAuctionIsNotHidden`	Hiding expires with the auction, so a concluded blind auction still reveals its winning bid	Guarded watchlist and bid-history queries	Inspect each guard predicate	Every guard is conditional on `date_end > CURRENT_TIMESTAMP`, so it cannot outlive the auction	Boundary

Blind auctions: auto-bid does not apply

Proxy bidding counter-bids one increment above the visible leader. A sealed auction has neither a visible leader nor a moving price, so auto-bid cannot work on one. It is now refused explicitly instead of being accepted and silently ignored. See Section 5.2.

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BLD-021	`TestBlindAuction.settingAnAutoBidIsRefused`	Setting an auto-bid on a blind auction is refused, with a reason the buyer can act on	Signed-in buyer, live blind auction	`POST /api/auto-bid` action `SET`, max $500	400, nothing stored, and the message explains that a sealed auction takes one hidden bid instead	Negative
TC-BLD-022	`TestBlindAuction.cancellingIsStillAllowed`	Cancelling is still permitted, so a row created before the guard can be cleared	Blind auction carrying a legacy auto-bid row	`POST /api/auto-bid` action `CANCEL`	200, and the row is deleted	Functional
TC-BLD-023	`TestBlindAuction.readingOneBackReportsNone`	Reading an auto-bid back on a blind auction reports none, whatever is still stored	Blind auction with a surviving legacy row	`GET /api/auto-bid?auctionId=...`	404 "No auto-bid set.", and the stored row is not even read	Negative
TC-BLD-024	`TestBlindAuction.openBlindDoesNotEchoAnAutoBid`	The detail payload does not advertise an auto-bid that will never fire	Live blind auction, and the viewer has a legacy $900 auto-bid row	`GET /api/auction/{id}` as that buyer	No `myAutoBid` field, so no "Auto-Bid Active" panel promising a defence that does not exist	Security
TC-BLD-025	`TestBlindAuction.anAscendingAuctionStillAcceptsOne`	The restriction is type-specific and has not broken ordinary auto-bidding	Signed-in buyer, live **ascending** auction	`POST /api/auto-bid` action `SET`, max $500	200, and the auto-bid is stored as before	Functional

3.9 Section 8: Auto-Bid and Proxy Bidding

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-AUT-001	`TestSetAutoBidServlet.successfulSet`	A buyer can register a proxy-bid ceiling	Auction live, and the buyer is not the seller	Valid maximum amount	Auto-bid stored encrypted and confirmed	Functional

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-AUT-002	`TestSetAutoBidServlet.competingHigherMaxWins`	Two competing auto-bidders resolve in favour of the higher ceiling	Two auto-bids on the same auction	Trigger the proxy round	The higher ceiling leads at one increment above the loser's maximum	Functional
TC-AUT-003	`TestSetAutoBidServlet.equalMaxFifoTiebreak`	Equal ceilings are resolved deterministically by who registered first	Two auto-bids with identical maxima	Trigger the proxy round	The earlier registration wins. The order is FIFO, not arbitrary	Boundary
TC-AUT-004	`AutoBidDAOStaleKeyTest.undecryptableRowReadsAsAbsent`	A ceiling encrypted under a rotated key degrades safely	Stored auto-bid ciphertext not decryptable with the current key	Read the auto-bid	Treated as absent rather than crashing or bidding a wrong amount	Negative

Auto-bid does not apply to blind auctions and is now refused on one. Those cases live with the rest of the blind-auction suite as TC-BLD-021 to TC-BLD-025 in Section 3.8.

3.10 Section 9: Buyer Engagement (Watchlist, Q&A, Ratings, Reports)

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-BUY-001	`TestWatchlistServlet.addSuccess`	A buyer can add a listing to their watchlist	Authenticated buyer, and the auction is not already watched	`action=add` with a valid `auctionId`	Row inserted and success confirmed	Functional
TC-BUY-002	`TestWatchlistApiServlet.getForbiddenForGuest`	An unregistered visitor cannot read a watchlist	No session	`GET /api/watchlist`	401, and no data is returned	Security
TC-BUY-003	`TestAuctionQuestionServlet.testAskSuccessSanitized`	A buyer can ask the seller a question, and the text is sanitised	Authenticated buyer, and the auction is not their own	Question text containing markup	Question stored sanitised, and the seller is notified	Functional
TC-BUY-004	`TestAuctionQuestionServlet.testReplyWrongSeller403`	Only the owning seller can answer a question on their listing	Authenticated seller who does not own the auction	Reply payload	403, and no answer is stored	Security
TC-BUY-005	`TestRateSellerServlet.buyerNotWinner`	Only the winning buyer may rate the seller	Auction finished, and the session user did not win	Score of 5	Rating refused	Security
TC-BUY-006	`TestRateSellerServlet.scoreBoundaryMax`	The rating scale accepts its maximum value	Winning buyer, auction finished, not yet rated	Score of 5	Accepted and stored	Boundary
TC-BUY-007	`TestBuyerReportServlet.duplicateReport`	A buyer cannot report the same listing twice	An open report already exists from this buyer	Second report submission	Refused with a message saying it was already reported	Negative

3.11 Section 10: Orders, Payment, Shipping and Refunds

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ORD-001	`TestOrderApiServlet.paySuccess`	A winning buyer can pay for an order with a saved payment method	Unpaid order owned by the session user	`action=pay` with their own payment method id	Order marked paid, and both parties notified	Functional
TC-ORD-002	`TestOrderApiServlet.payRejectsForeignMethod`	A buyer cannot pay using another user's stored payment method	Payment method belongs to a different user	`action=pay` with that method id	Rejected, and the order remains unpaid	Security

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ORD-003	`TestOrderApiServlet.shippingRejectsUnpaidOrder`	Shipping cannot be advanced before payment	Order is unpaid	Seller advances the shipping stage	Refused, and the stage is unchanged	Negative
TC-ORD-004	`TestOrderApiServlet.refundReasonTooShort`	A refund request must carry a meaningful reason	Paid order owned by the session user	Refund reason below the minimum length	400 validation error, and no refund is raised	Negative
TC-ORD-005	`OrderDAOPaymentTimeoutTest.overdueOrderIsCancelled`	Unpaid orders past the payment deadline are auto-cancelled	Unpaid order older than the configured deadline	Run the timeout sweep	Order cancelled, and the query never touches paid orders or the auction tables	Integration
TC-ORD-006	`AccountApiPaymentMethodUpdateTest.updateRefusesPanChange`	A stored card number cannot be altered through the edit endpoint	Saved card owned by the session user	Update payload including a different PAN	PAN change refused, and only holder and expiry are editable	Security

3.12 Section 11: Seller (Listings, Maintenance and Analytics)

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-SEL-001	`TestCreateAuctionServlet.TestValidTags`	A seller can publish a listing with images and tags	Session user has the selling capability	Complete valid listing form	Auction, detail, image and tag rows created in one transaction	Functional
TC-SEL-002	`TestCreateAuctionServlet.TestNotSeller`	A member without the selling capability cannot create a listing	Session user is a plain buyer	Valid listing form	Refused before any write	Security
TC-SEL-003	`TestCreateAuctionServlet.testInvalidExtension`	Only permitted image types can be uploaded	Seller session	File with a disallowed extension	Upload rejected, and no file is written to disk	Security
TC-SEL-004	`TestCreateAuctionServlet.testFileTooLarge`	Oversized uploads are rejected at the boundary	Seller session	Image exceeding the size cap	Rejected with a size error	Boundary
TC-SEL-005	`TestEditAuctionServlet.hasBidsBlocksEdit`	A listing that already has bids cannot be edited	Auction owned by the seller with at least one bid	Edit submission	Edit refused, which protects bidders who bid on the original terms	Negative
TC-SEL-006	`TestEditAuctionServlet.toctouBidsPlacedAfterGet`	A bid placed between loading and submitting the edit form is still caught	Zero bids when the form loaded, and a bid arrives before submit	Edit submission	Re-checked at write time and refused, so there is no time-of-check to time-of-use hole	Security
TC-SEL-007	`TestSellerListingMaintenance.lastUnitEndsListingAndNotifies`	Removing the final unit of stock closes the listing and tells the bidders	Listing with quantity 1 and existing bidders	Remove one unit with a reason	Listing ended, reason persisted, and every bidder notified	Integration

3.13 Section 12: Admin Console and Database Backup/Restore

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ADM-001	`TestAdminManageUserServlet.TestSuspend`	An admin can suspend a member account	ADMIN session, and the target is an active member	`action=suspend` with the target user id	Account status set to SUSPENDED	Functional
TC-ADM-002	`TestAdminManageUserServlet.TestSelfAction`	An admin cannot moderate their own account	ADMIN session	Target user id equals their own id	Action refused	Negative

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ADM-003	`TestAdminManageUserServlet.TestUnbanAdminTarget`	One admin cannot alter another admin's account status	ADMIN session, and the target is another ADMIN	`action=active`	Refused, because admin accounts are not moderatable through this path	Security
TC-ADM-004	`TestAdminReportServlet.testGetNonAdmin`	A non-admin cannot read the report moderation queue	Session holds a BUYER	`GET` the admin reports endpoint	Access denied, and no report data is returned	Security
TC-ADM-005	`TestAdminCategoriesServlet.testDeleteRestrictedByAuctions`	A category still in use cannot be deleted	Category referenced by live auctions	`action=DELETE`	Deletion refused with a referential-integrity message	Negative
TC-ADM-006	`DatabaseBackupUtilTest.rejectDestructive`	An uploaded restore file cannot smuggle in destructive SQL	ADMIN performing a restore	Backup file containing `DROP` or `DELETE` statements	File rejected before execution	Security
TC-ADM-007	`DatabaseBackupUtilTest.allowsSemicolonInAValue`	The restore parser is not defeated by a semicolon inside a legitimate data value	None	`INSERT` whose value contains `;`	Statement parsed as one insert and accepted, which shows the guard is precise rather than naive	Boundary

3.14 Section 13: Recommendation Engine

Addresses assessor feedback that recommendations must be explainable.

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-REC-001	`TestRecommendationApiServlet.personalisedForBuyer`	A signed-in buyer receives personalised recommendations	Buyer with interaction history	`GET /api/recommendations`	Personalised result set flagged as personalised	Functional
TC-REC-002	`TestRecommendationApiServlet.trendingForAnonymous`	An unregistered visitor still receives useful recommendations	No session	`GET /api/recommendations`	Trending listings returned, explicitly not marked personalised	Functional
TC-REC-003	`TestRecommendationPipeline.coldStartFallsThroughToTrending`	A brand-new user with no history is not shown an empty rail	User with zero interactions	Run the pipeline	Trending candidates fill the rail	Negative
TC-REC-004	`TestRecommendationPipeline.categoryCapLimitsASingleCategory`	The diversity cap stops one category dominating the rail	Candidate pool skewed to one category	Run the re-ranker	No category exceeds the configured cap, and the page is never shortened to achieve it	Functional
TC-REC-005	`TestRecommendationPipeline.recencyMultiplierFadesWithAge`	Older interactions count for less than recent ones	Interactions of varying age	Score the candidates	The recency multiplier decays with age, and future-dated timestamps cannot amplify a score	Boundary
TC-REC-006	`TestRecommendationProvenance.withholdsMaskedNameForLoneClicker`	Explainability copy never de-anonymises a single other user	Exactly one other user interacted with the item	Build the "why this?" provenance	The reason stays generic, with no masked name that identifies one person	Security

3.15 Section 14: Notifications and Preferences

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-NOT-001	`BidApiNotificationTest.plainBidNotifiesThePreviousLeader`	The outbid buyer is told when they lose the lead	Auction with an existing leading bidder	A higher bid from another buyer	Exactly one outbid notification, sent to the displaced leader only	Functional

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-NOT-002	`NotificationServiceLostTest.everyLoserExceptTheWinner`	Every losing bidder is told the auction ended, and the winner is not	Auction ended with several bidders	Finalise the auction	All losers notified once each, and the winner is excluded by the query itself	Functional
TC-NOT-003	`NotificationServiceLostTest.bodyNeverNamesTheWinner`	A losing-bidder notification does not disclose who won	Auction ended	Finalise the auction	The message body contains no winner identity	Security
TC-NOT-004	`NotificationServiceSellerAlertsTest.aBiddingWarCollapsesIntoOneMessage`	Rapid bidding does not spam the seller	Many bids in quick succession	Run the alert path	Bids coalesce into a single message carrying the current price	Functional
TC-NOT-005	`TestUpdatePreferenceServlet.testAllFalse`	A member can switch every optional notification off	Authenticated session	All preference toggles set false	Preferences persisted, and subsequent optional pushes are suppressed	Functional

3.16 Section 15: Telegram Notification Integration

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-TEL-001	`TelegramApiServletTest.linkStartMintsBothPaths`	A member can start linking their Telegram account	Authenticated session, and the bot is configured	`POST` link-start	A deep link and a one-time code are both issued	Functional
TC-TEL-002	`TelegramWebhookServletTest.wrongSecretIsRejected`	The webhook only accepts calls carrying the correct secret token	Bot configured	Webhook call with a wrong secret, including one that is a prefix of the correct value	Rejected, and no state changes	Security
TC-TEL-003	`TelegramWebhookServletTest.bruteForceIsBlocked`	Link codes cannot be brute-forced through the bot	Chat has made repeated wrong attempts	Further wrong codes	Chat blocked after the attempt limit	Security
TC-TEL-004	`TelegramLinkDAOTest.consumeCode_isSingleUse`	A link code cannot be redeemed twice	Valid unconsumed code	Redeem the code twice	The first redemption succeeds and the second returns nothing, because consumption is atomic and expiry-checked in one statement	Security
TC-TEL-005	`TelegramOutboxWorkerTest.blockedUserDeactivatesTheLink`	A user who blocks the bot stops receiving pushes	Outbox message for a chat that has blocked the bot	Worker drains the outbox	Link deactivated, and there is no infinite retry loop	Negative

3.17 Section 16: Account Management and PDPA

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ACC-001	`TestUpdateProfileServlet.update_reencryptsPiiBeforePersist`	Phone and address are encrypted at rest when a profile is updated	Authenticated session	New phone and address values	Values encrypted via `SecurityUtil` before the DAO write, and plaintext is never persisted	Security
TC-ACC-002	`TestAccountManagementServlet.loadsOnlySessionUserIdIgnoresRequestParam`	A member cannot load someone else's account page by changing a URL parameter	Authenticated session	`?userId=` set to another user's id	The session user's own record is loaded, and the parameter is ignored	Security

ID	Test class and method	Objective	Precondition	Input	Expected result	Type
TC-ACC-003	`UserDAOClosureTest.paidUndespatchedSaleRaisesRefund`	Closing an account does not strand a buyer who already paid	Closing seller has a paid, un-despatched sale	Execute account closure	A refund decision is raised for that order, and admins are alerted	Integration
TC-ACC-004	`UserDAOClosureTest.cleanupFailureRollsBackAnonymisation`	Account closure is all-or-nothing	Closure cleanup step fails mid-way	Execute account closure	The whole transaction rolls back, so there is no half-anonymised account	Integration

3.18 Frontend (React SPA)

All frontend cases run under `npm test`, using Vitest with React Testing Library.

ID	Test file and test name	Objective	Precondition	Input	Expected result	Type
TC-FE-001	`ProtectedRoute.test.jsx`, *"sends a signed-out visitor to the login page"*	Guarded pages are unreachable without a session	No authenticated user in context	Navigate to a protected route	Redirected to `/login`	Security
TC-FE-002	`ProtectedRoute.test.jsx`, *"offers the selling gate to a member who has not switched selling on"*	A buyer without the selling capability is guided rather than shown an error	Signed-in member, selling not enabled	Navigate to a seller route	The selling opt-in gate is rendered	Functional
TC-FE-003	`AuctionCard.test.jsx`, *"only shows the 'why this?' panel for a recommended card"*	Recommendation explainability appears only where a recommendation was actually made	One recommended card and one ordinary card	Render both	Explain panel present on the recommended card only	Functional
TC-FE-004	`AuctionCard.test.jsx`, *"shows the re-ranker score and the component behind it once expanded"*	The user can see why an item was recommended	Recommended card carrying provenance	Expand the "why this?" panel	The score and the dominant scoring component are shown	Functional
TC-FE-005	`CountdownTimer.test.jsx`, *"colours by urgency: normal, then amber inside a day, then red in the last hour"*	Time pressure is communicated visually at the right thresholds	Auctions ending in more than a day, less than a day, and less than an hour	Render each	Neutral, amber and red styling respectively	Boundary
TC-FE-006	`Modal.test.jsx`, *"moves focus to the first focusable element on open"*	Dialogs are usable by keyboard and screen reader	Modal closed	Open the modal	Focus moves into the dialog, and is restored to the trigger on close	Functional
TC-FE-007	`AccountSettings.test.jsx`, *"does not offer the card number for editing, and says why"*	The UI mirrors the server rule that a PAN is immutable	Saved card method	Open the card editor	No card-number field, and an explanation is shown	Security
TC-FE-008	`usePolling.test.jsx`, *"stops polling and aborts the in-flight request on unmount"*	Live-price polling does not leak timers or requests	Component polling an endpoint	Unmount the component	Interval cleared and the in-flight request aborted	Negative

4. Traceability Note

Each ID above resolves to a single named method in a single named class, so an assessor can verify any row by running:

```
cd FYP
mvn -B test -Dtest=TestPlaceBidServlet # a whole class
mvn -B test -Dtest=TestPlaceBidServlet#selfBidRejected # a single case
mvn -B test -Dtest=TestBlindAuction # all 53 blind-auction cases
```

`TestBlindAuction` and several other classes group their cases with JUnit 5 `@Nested`. To run one method inside a group, qualify it with the group name, for example `-Dtest='TestBlindAuction$SealedDetail#openBlindHidesTheStandingBidFromRivalBidders'`.

```
cd FYP/Frontend
npx vitest run src/components/ProtectedRoute.test.jsx
```

5. Coverage Gaps and Mitigations

The following areas have thin or no automated coverage. Each is stated with its mitigation so the position is defensible under questioning.

ID	Gap	Risk	Mitigation
G1	Blind (sealed-bid) auction behaviour. **This gap is now closed.** 58 automated tests were added, 53 in `TestBlindAuction` and 5 in `AuctionEventPublisherBlindTest`. They cover the confidentiality guard on every read path, the one-bid-per-buyer rule, rejection of Buy It Now, Dutch acceptance and auto-bid, and winner determination with order creation at close. Writing them uncovered five real confidentiality defects, all since fixed and pinned by regressions. See Section 3.8 for TC-BLD-001 to TC-BLD-025, and Section 5.1.	Closed	None needed
G2	The auction detail endpoints `AuctionApiServlet` and `AuctionDetailServlet` have no direct test class.	Medium	The behaviour they compose *is* tested through `TestAuctionBidHistory`, `TestAuctionQuestionServlet`, `TestBidApiServlet`, `TestWatchlistApiServlet` and `TestRecommendationApiServlet`. The page itself is exercised by manual end-to-end testing.
G3	`AdminApiServlet`, the SPA-facing admin API covering users, listings, categories, reports, reviews, orders, analytics, audit log, database backup/restore and recommendation settings, has no direct test class.	Medium	Every business rule it enforces is covered one layer away, in `TestAdminManageUserServlet`, `TestAdminAuctionServlet`, `TestAdminCategoriesServlet`, `TestAdminReportServlet`, `TestAdminListingsServlet`, `AdminManagementDAOTest`, `UserDAOAdminLookupTest`, `DatabaseBackupUtilTest`, and recommendation-settings validation in `TestRecommendationPipeline.weightsAreClamped`.
G4	Google OAuth sign-in (`OAuthApiServlet`) has no automated tests.	Medium	Not testable without mocking Google's token endpoint. Verified by manual test: sign in with a Google account, then confirm a linked account row is created and the session role is correct.
G5	Support chat (`SupportApiServlet`, `SupportChatDAO`) and order messaging (`OrderMessageApiServlet`, `OrderMessageDAO`) have no automated tests.	Low to medium	Both are auxiliary communication features. Covered by manual testing of the buyer-to-seller and member-to-admin conversation flows.
G6	Real-time price updates over server-sent events (`AuctionEventServlet`, `AuctionEventBus`) have no automated tests.	Low	Timing-dependent transport that is impractical to unit test. The polling fallback on the client *is* tested, in `usePolling.test.jsx` with 7 tests. Verified manually with two browsers on one auction.

ID	Gap	Risk	Mitigation
G7	Frontend page components. Only 3 of 21 pages have tests: `AccountSettings`, `CreateAuction` and `MyListings`. `AuctionDetail`, `Search`, `Home`, `Login`, `Register`, `Watchlist`, `MyPurchases` and all 13 admin pages are untested. Utilities, hooks and the API layer are well covered.	Medium	The business logic those pages render is tested on the server, and the shared building blocks they compose (`AuctionCard`, `CountdownTimer`, `Modal`, `ProtectedRoute`, `usePolling`, `orders.js`, `listings.js`, `helpers.js`) are tested in isolation. Page-level behaviour is covered by the manual test script.
G8	Minor untested classes: `FeaturedApiServlet` with `FeaturedListingDAO`, `PlatformStatsApiServlet`, `ProfilePhotoApiServlet`, `UploadedFileServlet`, `HealthApiServlet`, `BrowseHistoryDAO`, `PlatformRevenueDAO`, `LinkedAccountDAO`, `AdminReportDAO`.	Low	Thin read-only or pass-through components. Exercised indirectly whenever the landing page, admin dashboard or profile pages are loaded during manual testing.
G9	Live-database integration tests are skipped by default. This is 6 tests across `AdminManagementDAOIntegrationTest` and `DatabaseBackupRestoreIntegrationTest`.	Low	Intentional, so the build stays hermetic. Run on demand with `AUCTION_DB_IT=true mvn -B test` against a real PostgreSQL instance. This was executed before submission.

5.1 Blind Auction Confidentiality Defects (All Fixed)

This section is closed and nothing in it is outstanding. An earlier draft listed three open defects, D1 to D3, and asked for a decision before the presentation. That decision was taken. All three were fixed, a further sweep found two more of the same kind (D4 and D5) which were fixed alongside them, and every one now has a regression test that was confirmed to fail against the pre-fix code. The fixes are in commit `2d3a234`, "Close every path that leaked a live sealed bid, and refuse auto-bid on one". This section is safe to finalise as written.

Writing TC-BLD-001 to TC-BLD-008 meant auditing the read paths that can reach a live blind auction's leading bid. The guard turned out to be missing in two DAOs that the first pass had not examined, which prompted a second and exhaustive sweep of every DAO and servlet projecting a price or bid amount. That sweep is recorded in Section 5.1.1. Five paths were defective:

ID	Defect	Exposure	Fix applied	Regression
D1	`WatchlistDAO.listByUser` computed `current_bid` from `MAX(bid_amount)` with no blind guard, and `GET /api/watchlist` returned it. Watchlisting a sealed auction therefore read out its leading bid.	**High.** Any signed-in buyer, through normal UI. It defeated the mechanism outright: watchlist the item, read the top bid, then bid one dollar more.	The guard that resolves `auction_type = 3` to `starting_price`, as already used in `SearchDAO`, `RecommendationDAO` and `FeaturedListingDAO`. It is conditional on the auction still running, so a concluded one still reveals its winning bid.	TC-BLD-015
D2	`SellerProfileDAO.getActiveListings` projected `current_price` from `MAX(bid_amount)` with no blind guard, on a query already filtered to live auctions.	**High.** Anyone at all, including an unregistered visitor with no session.	The same guard.	TC-BLD-016
D3	The legacy JSP endpoints served full bid amounts for a live blind auction on a direct request. Two servlets were affected rather than one: `AuctionBidHistoryServlet` (`GET /auction-bids`) and `AuctionDetailServlet` (`GET /auction/{id}`). The SPA uses the correctly-guarded `/api/auction/{id}/bids` and was never affected.	**Medium.** Not reachable through the SPA, but reachable by anyone who types the URL.	Guarded inside `BidDAO.getBidHistory`, in both overloads, rather than in either servlet. The reasoning is set out below.	TC-BLD-017
D4	`BidDAO.placeBid` had no auction-type guard, and the legacy `POST /protected/bid` servlet calls it for every type. On a sealed auction it compared the bid against `MAX(bid_amount)`, so the `BID_TOO_LOW` rejection answered the question *"is the top sealed bid above X?"* for any X. A handful of probes recovers the leading bid exactly. It also admitted extra bids past the one-per-buyer rule, and fired outbid and new-bid notifications carrying the amount.	**High.** This is a price oracle. It is also the only defect here that leaks through an error message rather than a payload, which is why the first pass over the read paths missed it.	`placeBid` now rejects `BLIND` with `WRONG_AUCTION_TYPE` before any comparison. That joins the guards `acceptDutchBid`, `buyItNow` and `placeSealedBid` were already carrying.	TC-BLD-018
D5	`SearchDAO` guarded the price column on the result page but not on the count query behind it. Narrowing `minPrice` and `maxPrice` and watching the result total move located a sealed bid that never appeared on screen. It also made the count disagree with the page it counted.	**Medium.** An inference oracle available to any visitor.	The count query now uses the same `SEALED_SAFE_PRICE` column as the result page.	TC-BLD-019

On D3, the choice was between suppressing the amounts and retiring the `/auction-bids` mapping. Neither servlet was retired. Both still forward to JSP views that are part of the deployed application, and deleting a route days before the demo is a worse trade than adding a `WHERE` clause. Putting the guard in `BidDAO.getBidHistory` also covers more ground: it fixed `AuctionDetailServlet`, a second unguarded caller found during the sweep, and it covers any future caller written by someone who does not know the rule. `AuctionApiServlet` keeps its own short-circuit, so the SPA path is now guarded twice.

5.1.1 The Audit in Full

Every DAO and servlet that projects a price or a bid amount, with the reason each is safe. This list is exhaustive. It is the complete set of matches for `bid_amount`, `current_price`, `current_bid` and `winning_bid` across `FYP/src/main/java`.

Read path	Verdict
`SearchDAO`, result page	Guarded, via `SEALED_SAFE_PRICE`
`SearchDAO`, price-filtered count	Was D5, now guarded
`RecommendationDAO`, 5 projections	Guarded
`FeaturedListingDAO`	Guarded
`WatchlistDAO.listByUser`	Was D1, now guarded
`SellerProfileDAO.getActiveListings`	Was D2, now guarded
`BidDAO.getBidHistory`, both overloads	Was D3, now guarded
`BidDAO.placeBid`	Was D4, now rejects blind
`AuctionApiServlet`, detail and bid history	Guarded. The seller and self-bid exceptions are deliberate
`AuctionEventPublisher.buildSnapshot`, SSE	Guarded
`BidDAO.getUserBidAmount`	Safe. Returns the caller's own bid only
`ProfileActivityDAO.getBidHistory`	Safe. It reads the auction top bid but uses it only in Java to compute `won`, which additionally requires the auction to have ended. The amount is not carried on the row
`ProfileActivityDAO` transaction volume	Safe. Sums `winning_bid`, which exists only after close
`SellerAuctionDAO`, listing rows, sorts and `getBidHistory`	Safe. Every query is scoped to `seller_id`, and a seller may see bids on their own listing
`SellerAnalyticsDAO`	Safe. Scoped to `seller_id`
`AuctionDAO.getAllAuctions`	Safe. Admin-only moderation view
`AuctionDAO` revenue sums, and `AdminReportDAO`	Safe. Admin-only, and `winning_bid` exists only after close
`OrderDAO`, in `declareWinner` and `ensureOrderForAuction`	Safe. Runs at or after close, when the amount is public
`AuctionFinalizer`	Safe. Runs at close
`AutoBidDAO` top-bid reads	Safe. Internal to the bidding transaction, and unreachable for blind now that `placeBid` rejects it
`NotificationService`, in `auctionSummary` and `sellerBidSnapshot`	Safe. The two callers that run while an auction is live, `notifyOutbid` and `notifySellerNewBid`, are on the ascending path only, which D4's fix confirms. The rest run at close

No sixth leak was found. The two paths missed on the first pass were both listing projections in DAOs that the first sweep did not open at all. That is why the second sweep was driven by a text search across the whole source tree instead of by a list of endpoints.

5.2 Behaviour That Is Ambiguous or Unimplemented

Recorded so it can be answered confidently rather than guessed at.

Auto-bid does not apply to blind auctions, and now says so. This was previously listed here as ambiguous, and it is now resolved. `BidApiServlet` routes a blind auction to the sealed path, which never consults the auto-bid table, so proxy bidding never had any effect. `AutoBidApiServlet` still accepted a ceiling for one, though, and the detail payload echoed it back as `myAutoBid`. The result was a buyer being shown an "Auto-Bid Active" panel on an auction where nothing was bidding for them. `AutoBidApiServlet` now rejects it with a 400 explaining why, `GET` reports none, the detail payload omits `myAutoBid`, and the sealed-bid form on `AuctionDetail.jsx` says in one line why auto-bid is unavailable. Cancelling is still allowed so a pre-existing row can be cleared. This is covered by TC-BLD-021 to TC-BLD-025. A cleanup migration, `migration_blind_auction_no_auto_bid.sql` registered in `migrate_all.sql`, deletes the rows that had been stored against blind auctions. They can never fire, nothing reads them any more, and each holds an encrypted maximum the buyer can no longer see or withdraw. No database constraint accompanies it, because the rule refers to another table and a PostgreSQL `CHECK` cannot do that.

The tie-break is decided by the database, not by Java. The winner query orders by `bid_amount DESC, bid_time ASC`, so the earliest of two identical bids wins. TC-BLD-014's companion test pins the query shape. The suite has no database, so the outcome itself is not proven by an automated test.

A blind auction has no reserve behaviour of its own. `reservePrice`, stored as `max_price`, is carried on the payload but nothing in the sealed path consults it. A blind auction therefore concludes at its top bid regardless of the reserve.

The sealed path is not rate limited. `placeSealedBid` does not call the bid rate limiter that `placeBid` uses. This is harmless because one bid per buyer is enforced instead, but it is a real difference if asked.

5.3 Recommended Action Before the Presentation

1. D1 to D5 need no further action. All are fixed, and all have regressions confirmed to fail against the pre-fix code. This is worth rehearsing as a positive: the audit was prompted by writing tests, it found five real confidentiality defects in a headline feature, and each fix is pinned by a test that reproduces the original bug.
2. Prepare a one-slide answer for G4 (Google OAuth) and G7 (frontend page tests), framed as deliberate scope decisions with manual coverage rather than as oversights.

6. Appendix: Full Backend Test Class Inventory

Grouped by section, with declared test-method counts. The totals sum to 1,443.

1. Guest landing and dynamic content (27 tests) `TestAdminLandingContentApiServlet` 15, `TestLandingContentApiServlet` 4, `TestSpaFallbackFilter` 4, `TestAuctionTagDAO` 3, `TestCategoryApiServlet` 1

2. Search, filters, sorting and browse (80 tests) `TestSearchServletFilters` 31, `TestSearchServletSort` 25, `TestSearchServlet` 16, `TestSearchServletCategory` 6, `TestSearchApiServlet` 2

3. Auction detail and public seller profile (51 tests) `TestAuctionStateUtil` 23, `TestAuctionBidHistory` 12, `TestSellerProfileServlet` 9, `TestAuctionDAO` 7

4. Registration and login (58 tests) `TestMergedSellerAccount` 17, `TestAuthApiServlet` 15, `TestLoginServlet` 9, `TestRegisterServlet` 9, `LoginAttemptLimiterTest` 8

5. Password reset, OTP and 2FA (57 tests) `TestTwoFactorServlet` 22, `TestPasswordResetServlet` 12, `OtpStoreTest` 9, `TestChangePasswordServlet` 6, `TokenStoreTest` 5, `TestTwoFactorApiServlet` 3

6. Session, RBAC, filters and platform security (43 tests) `SecurityUtilTest` 10, `InputValidatorProfileFieldsTest` 10, `TestLogoutServlet` 9, `TestAdminFilter` 5, `TestSecurityFilter` 3, `TestCorsFilter` 2, `TestSessionApiServlet` 2, `DevModeTest` 2

7. Bidding, covering ascending, Dutch, Buy It Now and blind (130 tests) `TestBlindAuction` 53, `TestPlaceBidServlet` 19, `TestWinningBidPrecision` 10, `DutchClockTest` 9, `TestBiddingHistoryServlet` 9, `TestBidApiServlet` 8, `BidOutcomeTest` 8, `BidDAORateLimitTest` 6, `AuctionEventPublisherBlindTest` 5, `AuctionTypeTest` 3

8. Auto-bid and proxy bidding (45 tests) `TestSetAutoBidServlet` 27, `TestAutoBidApiServlet` 9, `AutoBidOutbidNotificationTest` 4, `AutoBidDAOStaleKeyTest` 3, `TestAutoBidSelfBidGuard` 2

9. Buyer engagement, covering watchlist, Q&A, ratings and reports (139 tests) `TestWatchlistServlet` 25, `TestAuctionQuestionServlet` 21, `TestSellerRateBuyerServlet` 21, `TestRateSellerServlet` 18, `TestBuyerReportServlet` 16, `TestWatchlistApiServlet` 10, `TestReportUserServlet` 9, `TestQuestionApiServlet` 6, `ReportDAOReplyTest` 6, `TestRateApiServlet` 4, `TestReportApiServlet` 3

10. Orders, payment, shipping and refunds (57 tests) `AccountApiPaymentMethodUpdateTest` 20, `TestOrderApiServlet` 11, `PaymentMethodDAOUpdateTest` 8, `OrderDAOPaymentTimeoutTest` 7, `PaymentMethodTest` 7, `AuctionExpiryListenerPaymentTimeoutTest` 3, `TestOrderDAOLabels` 1

11. Seller listings, maintenance and analytics (215 tests) `TestSellerAuctionDAO` 67, `TestSellerListingMaintenance` 39, `TestSellerDashboardServlet` 23, `SellerAnalyticsEmailTest` 23, `TestCreateAuctionServlet` 16, `TestEditAuctionServlet` 15, `SellerApiListingKindTest` 12, `TestCancelAuctionServlet`

11, `ListingKindTest` 9

12. Admin console and database backup/restore (133 tests) `AdminManagementDAOIntegrationTest` 25 (environment-gated), `TestAdminCategoriesServlet` 19, `AdminManagementDAOTest` 15, `TestAdminReportServlet` 15, `TestAdminAuctionServlet` 13, `DatabaseBackupUtilTest` 12, `TestAdminDashboardServlet` 11, `TestAdminManageUserServlet` 11, `DatabaseBackupRestoreIntegrationTest` 5 (environment-gated), `UserDAOAdminLookupTest` 4, `TestAdminListingsServlet` 3

13. Recommendation engine (104 tests) `TestRecommendationPipeline` 43, `UserBasedCollaborativeFilterTest` 31, `TestRecommendationApiServlet` 18, `TestRecommendationProvenance` 12

14. Notifications and preferences (93 tests) `NotificationServiceOrderAlertsTest` 28, `NotificationServiceSellerAlertsTest` 24, `NotificationServiceLostTest` 9, `TestUpdatePreferenceServlet` 8, `BidApiNotificationTest` 7, `TestViewNotificationHistoryServlet` 6, `NotificationServiceOrderTimeoutTest` 5, `AuctionCancelledPreferenceTest` 4, `TestNotificationApiServlet` 2

15. Telegram integration (121 tests) `TelegramOutboxDAOTest` 22, `TelegramSellerAlertsTest` 17, `TelegramWebhookServletTest` 16, `TelegramOutboxWorkerTest` 14, `TelegramAlertsTest` 13, `TelegramApiServletTest` 10, `TelegramLinkDAOTest` 9, `TelegramPriceCooldownTest` 9, `TelegramAuctionCancelledAlertTest` 7, `TelegramAttemptLimiterTest` 4

16. Account management and PDPA (90 tests) `TestUserDAO` 17, `AccountApiProfileUpdateTest` 14, `UserDAOClosureTest` 12, `TestAccountApiServlet` 8, `AccountApiClosureNotificationTest` 8, `TestUpdateProfileServlet` 6, `RelativeTimeTest` 6, `TestAccountManagementServlet` 5, `TestDeleteAccountServlet` 4, `StatusTest` 3, `UserDAODeleteAccountTest` 3, `TestEditProfileServlet` 2, `UserDAOMappingTest` 2

(`com/auction/test/ApiTestSupport.java` *is a shared test helper and declares no test methods.)*

7. Appendix: Frontend Test File Inventory

File	Tests	Area
`src/pages/seller/MyListings.test.jsx`	24	Seller listing management UI
`src/utils/orders.test.js`	24	Order bucketing, tabs, refs, headlines, date filters
`src/utils/listings.test.js`	17	Unsold versus cancelled logic, display price, bucket counts
`src/utils/helpers.test.js`	15	Countdown formatting, initials, role labels, unescaping, currency
`src/components/Modal.test.jsx`	13	Dialog accessibility, focus trap, scroll lock
`src/api/requestConfig.test.js`	11	Request wire format and abort-signal threading
`src/api/seller.test.js`	11	Seller API payloads, covering reason, quantity and cost price
`src/api/user.test.js`	11	Payment-method and profile API payloads
`src/components/AuctionCard.test.jsx`	10	Card rendering, countdown, recommendation explainability
`src/pages/AccountSettings.test.jsx`	10	Payment-method editing, account-closure copy
`src/components/ProtectedRoute.test.jsx`	9	Route guarding and role/capability gating
`src/pages/seller/CreateAuction.test.jsx`	8	Product versus service listing kind
`src/hooks/usePolling.test.jsx`	7	Polling lifecycle, visibility, abort on unmount
`src/components/CountdownTimer.test.jsx`	6	Countdown ticking, urgency colours, shared interval
`src/utils/apiError.test.js`	6	Error-message resolution
`src/utils/appBase.test.js`	5	Base-path resolution
`src/hooks/useDebouncedValue.test.jsx`	3	Search input debouncing
Total	**190**	

Document generated from the repository state on 6 August 2026. All counts verified by executing `mvn -B test` *and* `npm test`*.*